```
python3 --version

which python3

mkdir -p ~/AISC

cd ~/AISC

pwd

python -m pip install numpy pandas matplotlib scikit-learn scipy seaborn reportlab

python -c "import numpy, pandas, matplotlib, sklearn, scipy, seaborn, reportlab; print('ALL MAIN LIBRARIES WORKING SUCCESSFULLY')"

nano experiment7.py

python3 experiment7.py


sudo apt update

sudo apt install python3-full python3-venv python3-pip

python3 -m venv ~/aisc_env

source ~/aisc_env/bin/activate

python -m pip install numpy pandas matplotlib scikit-learn scipy seaborn reportlab



python3 your_code.py
```

Exp 7 – Associative Memory

Autoassociative Memory

```
import numpy as np

# 1. Store patterns
patterns = np.array([
    [1, 1, 1, -1, -1, -1],
    [-1, -1, 1, 1, 1, -1]
])

# 2. Initialize weight matrix
W = np.zeros((6, 6))

# 3. Hebbian learning
for p in patterns:
    W += np.outer(p, p)

# 4. Remove self connections
np.fill_diagonal(W, 0)

print("Weight Matrix:")
print(W)

# 5. Give noisy input
test = np.array([1, -1, 1, -1, -1, -1])

# 6. Recall pattern
state = test.copy()
```

```
for iteration in range(10):
    old_state = state.copy()

    for i in range(6):
        activation = np.dot(W[i], state)

        if activation >= 0:
            state[i] = 1
        else:
            state[i] = -1

    if np.array_equal(state, old_state):
        break

# 7. Display output
print("\nNoisy Input:", test)
print("Recalled Pattern:", state)
print("Original Pattern:", patterns[0])
print("Recall Successful:", np.array_equal(state, patterns[0]))
```

HetroAssociative Memory

```
import numpy as np

# 1. Input patterns
A = np.array([
    [1, 1, -1, -1],
    [-1, -1, 1, 1],
    [1, -1, 1, -1],
    [-1, 1, -1, 1]
])

# 2. Target patterns
B = np.array([
    [1, 1, -1],
    [-1, -1, 1],
    [1, -1, -1],
    [-1, 1, 1]
])

# 3. Initialize weight matrix
W = np.zeros((4, 3))

# 4. Hebbian learning
for i in range(len(A)):
    W += np.outer(A[i], B[i])

print("Weight Matrix:")
```

```
print(W)

# 5. Recall target patterns
print("\nRecall Results")

for i in range(len(A)):
    activation = A[i] @ W
    output = np.where(activation >= 0, 1, -1)

    print("\nInput:", A[i])
    print("Recalled Output:", output)
    print("Target Output:", B[i])
    print("Correct:", np.array_equal(output, B[i]))
```

BAM

```
import numpy as np

# 1. Input patterns (6 neurons)
A = np.array([
    [1, 1, 1, 1, 1, 1],
    [-1, -1, -1, -1, -1, -1],
    [1, 1, -1, -1, 1, 1],
    [-1, -1, 1, 1, -1, -1]
])

# 2. Target patterns (3 neurons)
B = np.array([
    [1, 1, 1],
    [-1, -1, -1],
    [1, -1, 1],
    [-1, 1, -1]
])

# 3. Initialize weight matrix
W = np.zeros((6, 3))

# 4. Hebbian learning
for i in range(len(A)):
    W += np.outer(A[i], B[i])

print("BAM Weight Matrix:")
```

```
print(W)

# 5. Forward recall: A -> B
print("\nForward Recall: A -> B")

for i in range(len(A)):
    output_B = np.sign(A[i] @ W)

    print("\nInput A:", A[i])
    print("Recalled B:", output_B)
    print("Target B:", B[i])
    print("Correct:", np.array_equal(output_B, B[i]))

# 6. Reverse recall: B -> A
print("\nReverse Recall: B -> A")

for i in range(len(B)):
    output_A = np.sign(B[i] @ W.T)

    print("\nInput B:", B[i])
    print("Recalled A:", output_A)
    print("Target A:", A[i])
    print("Correct:", np.array_equal(output_A, A[i]))
```

Hopfield

```
import numpy as np

# 1. Store patterns
patterns = np.array([
    [1, 1, 1, -1, -1, -1],
    [-1, -1, -1, 1, 1, 1]
])

# 2. Noisy test pattern
test = np.array([1, -1, 1, -1, -1, -1])

# 3. Initialize weight matrix
W = np.zeros((6, 6))

# 4. Hebbian learning
for p in patterns:
    W += np.outer(p, p)

# 5. Remove self-connections
np.fill_diagonal(W, 0)

print("Hopfield Weight Matrix:")
print(W)

print("\nSymmetric Matrix:", np.array_equal(W, W.T))
print("Diagonal:", np.diag(W))
```

```
# 6. Define energy function
def energy(state):
    return -0.5 * state @ W @ state

# 7. Start with noisy pattern
state = test.copy()

print("\nInitial State:", state)
print("Initial Energy:", energy(state))

# 8. Iterative neuron updates
for iteration in range(10):

    old_state = state.copy()

    for i in range(6):
        activation = np.dot(W[i], state)

        if activation >= 0:
            state[i] = 1
        else:
            state[i] = -1

    print("Iteration", iteration + 1, ":", state)
    print("Energy:", energy(state))

    # Stop if stable
    if np.array_equal(state, old_state):
        break
```

```
# 9. Display final result
print("\nFinal Recalled Pattern:", state)
print("Expected Pattern:", patterns[0])

print("Recall Successful:",
      np.array_equal(state, patterns[0]))
```

KMeans Clustering

```
import numpy as np
from sklearn.cluster import KMeans

# Sample dataset
X = np.array([
    [1, 2],
    [2, 3],
    [3, 2],
    [8, 8],
    [9, 8],
    [8, 9],
    [15, 15],
    [16, 15],
    [15, 16]
])

# Create K-Means model with 3 clusters
model = KMeans(n_clusters=3, random_state=42, n_init=10)

# Train the model and predict clusters
labels = model.fit_predict(X)

# Display results
print("Cluster Labels:")
print(labels)

print("\nCluster Centers:")
```

```
print(model.cluster_centers_)
```

Hierarchical Clustering

```
import numpy as np
import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.cluster import AgglomerativeClustering

# Sample dataset
X = np.array([
    [1, 2],
    [2, 3],
    [3, 2],
    [8, 8],
    [9, 8],
    [8, 9],
    [15, 15],
    [16, 15],
    [15, 16]
])

# Create hierarchical clusters
model = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward'
)

labels = model.fit_predict(X)
```

```
print("Cluster Labels:")
print(labels)

# Create linkage matrix for dendrogram
Z = linkage(X, method='ward')

# Plot dendrogram
plt.figure(figsize=(8, 5))
dendrogram(Z)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

Evaluation metrics

```
import numpy as np
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Dataset
X = np.array([
    [1, 2], [2, 3], [3, 2],
    [8, 8], [9, 8], [8, 9],
    [15, 15], [16, 15], [15, 16]
])

# K-Means
km = KMeans(n_clusters=3, random_state=42, n_init=10)
k_labels = km.fit_predict(X)

print("K-Means Results")
print("Silhouette Score:", silhouette_score(X, k_labels))
print("Davies-Bouldin Score:", davies_bouldin_score(X, k_labels))
print("Inertia:", km.inertia_)

# Hierarchical Clustering
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
h_labels = hc.fit_predict(X)

print("\nHierarchical Results")
print("Silhouette Score:", silhouette_score(X, h_labels))
print("Davies-Bouldin Score:", davies_bouldin_score(X, h_labels))
```

Exp-6 Cluster:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score
from scipy.cluster.hierarchy import linkage, dendrogram

# --------------------------------
# 1. Sample Dataset
# --------------------------------

X = np.array([
    [1, 2],
    [2, 3],
    [3, 2],
    [8, 8],
    [9, 8],
    [8, 9],
    [15, 15],
    [16, 15],
    [15, 16]
])

# --------------------------------
# 2. K-Means Clustering
# --------------------------------

km = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
k_labels = km.fit_predict(X)

print("========== K-MEANS CLUSTERING ==========")

print("Cluster Labels:")
print(k_labels)

print("\nCluster Centers:")
print(km.cluster_centers_)

# --------------------------------
# 3. Hierarchical Clustering
# --------------------------------

hc = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward'
)

h_labels = hc.fit_predict(X)

print("\n========== HIERARCHICAL CLUSTERING ==========")

print("Cluster Labels:")
print(h_labels)

# --------------------------------
# 4. Dendrogram Visualization
# --------------------------------
```

```
Z = linkage(X, method='ward')

plt.figure(figsize=(8, 5))
dendrogram(Z)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# --------------------------------
# 5. Evaluation Metrics
# --------------------------------

print("\n========== EVALUATION METRICS ==========")

# K-Means Evaluation
print("\nK-Means Results")

print("Silhouette Score:",
      silhouette_score(X, k_labels))

print("Davies-Bouldin Score:",
      davies_bouldin_score(X, k_labels))

print("Inertia:", km.inertia_)

# Hierarchical Evaluation
print("\nHierarchical Results")
```

```
print("Silhouette Score:",
      silhouette_score(X, h_labels))

print("Davies-Bouldin Score:",
      davies_bouldin_score(X, h_labels))
```

Exp-9 Mamdani Fuzzy

```
import numpy as np
import matplotlib.pyplot as plt

# --------------------------------
# 1. Define input and output ranges
# --------------------------------
error_universe = np.linspace(-10, 10, 401)
change_universe = np.linspace(-5, 5, 401)
output_universe = np.linspace(-100, 100, 1001)

# --------------------------------
# 2. Triangular Membership Function
# --------------------------------

def triangular_mf(x, a, b, c):
    x = np.asarray(x, dtype=float)
    membership = np.zeros_like(x)

    # Left shoulder
    if a == b:
        membership[x <= b] = 1.0
        mask = (x > b) & (x < c)
        membership[mask] = (c - x[mask]) / (c - b)

    # Right shoulder
    elif b == c:
        mask = (x > a) & (x < b)
        membership[mask] = (x[mask] - a) / (b - a)
```

```
        membership[x >= b] = 1.0

    # Normal triangle
    else:
        rising = (x >= a) & (x <= b)
        membership[rising] = (x[rising] - a) / (b - a)

        falling = (x > b) & (x <= c)
        membership[falling] = (c - x[falling]) / (c - b)

    return np.clip(membership, 0, 1)

# --------------------------------
# 3. Define Fuzzy Membership Sets
# --------------------------------

error_sets = {
    "N": triangular_mf(error_universe, -10, -10, 0),
    "Z": triangular_mf(error_universe, -10, 0, 10),
    "P": triangular_mf(error_universe, 0, 10, 10)
}

change_sets = {
    "N": triangular_mf(change_universe, -5, -5, 0),
    "Z": triangular_mf(change_universe, -5, 0, 5),
    "P": triangular_mf(change_universe, 0, 5, 5)
}

output_sets = {
    "Cooling": triangular_mf(output_universe, -100, -100, 0),
```

```
    "Maintain": triangular_mf(output_universe, -100, 0, 100),
    "Heating": triangular_mf(output_universe, 0, 100, 100)
}

# --------------------------------
# 4. Define Nine Fuzzy Rules
# --------------------------------

rules = [
    ("N", "N", "Cooling"),
    ("N", "Z", "Cooling"),
    ("N", "P", "Maintain"),

    ("Z", "N", "Cooling"),
    ("Z", "Z", "Maintain"),
    ("Z", "P", "Heating"),

    ("P", "N", "Maintain"),
    ("P", "Z", "Heating"),
    ("P", "P", "Heating")
]

# --------------------------------
# 5. Take Crisp Input Values
# --------------------------------

error = -8
change = 1
```

```
# --------------------------------
# 6. Fuzzification
# --------------------------------

error_membership = {
    "N": triangular_mf(np.array([error]), -10, -10, 0)[0],
    "Z": triangular_mf(np.array([error]), -10, 0, 10)[0],
    "P": triangular_mf(np.array([error]), 0, 10, 10)[0]
}

change_membership = {
    "N": triangular_mf(np.array([change]), -5, -5, 0)[0],
    "Z": triangular_mf(np.array([change]), -5, 0, 5)[0],
    "P": triangular_mf(np.array([change]), 0, 5, 5)[0]
}

print("Error Membership:", error_membership)
print("Change Membership:", change_membership)

# --------------------------------
# 7. Rule Evaluation and Inference
# --------------------------------

aggregated = np.zeros_like(output_universe)

for e_set, c_set, out_set in rules:

    # Calculate firing strength using MIN
    strength = min(
        error_membership[e_set],
```

```
        change_membership[c_set]
    )

    # Apply MIN implication
    rule_output = np.minimum(
        strength,
        output_sets[out_set]
    )

    # Aggregate using MAX
    aggregated = np.maximum(aggregated, rule_output)

print("\nRule evaluation completed.")

# --------------------------------
# 8. Defuzzification using Centroid
# --------------------------------

if np.sum(aggregated) != 0:

    crisp_output = np.sum(
        output_universe * aggregated
    ) / np.sum(aggregated)

else:
    crisp_output = 0

print("Final Control Output:", crisp_output)
```

```
# --------------------------------
# 9. Plot Membership Functions
# --------------------------------

plt.figure(figsize=(10, 5))

for name, membership in output_sets.items():
    plt.plot(output_universe, membership, label=name)

plt.fill_between(
    output_universe,
    aggregated,
    alpha=0.3,
    label="Aggregated Output"
)

plt.axvline(
    crisp_output,
    linestyle="--",
    label="Final Output"
)

plt.title("Mamdani Fuzzy Logic Controller")
plt.xlabel("Control Action")
plt.ylabel("Membership Degree")
plt.legend()
plt.grid()
plt.show()
```

Exp 5- Supervised

// Import dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load provided dataset
df = pd.read_csv("data.csv")

# Explore dataset
print(df.head())
print(df.shape)
print(df.info())
print(df.isnull().sum())
```

// Create own dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create our own dataset
data = {
    "Age": [25, 30, 45, 28, 50, 35, 24, 42, 31, 55,
            27, 39, 48, 26, 44, 33, 29, 52, 36, 41],

    "MonthlyIncome": [20000, 35000, 70000, 25000, 85000,
                      50000, 18000, 75000, 40000, 90000,
                      22000, 60000, 80000, 21000, 72000,
```

```
                45000, 30000, 95000, 55000, 68000],

    "YearsAtCompany": [1, 3, 15, 2, 20, 8, 1, 12, 4, 22,
                2, 10, 18, 1, 14, 6, 3, 25, 9, 13],

    "OverTime": ["Yes", "No", "No", "Yes", "No",
                "Yes", "Yes", "No", "Yes", "No",
                "Yes", "No", "No", "Yes", "No",
                "Yes", "Yes", "No", "No", "No"],

    "JobSatisfaction": [2, 3, 4, 1, 4, 3, 1, 4, 2, 4,
                1, 3, 4, 2, 3, 2, 1, 4, 3, 4],

    "Attrition": ["Yes", "No", "No", "Yes", "No",
                "No", "Yes", "No", "Yes", "No",
                "Yes", "No", "No", "Yes", "No",
                "Yes", "Yes", "No", "No", "No"]
}

# Convert dictionary into DataFrame
df = pd.DataFrame(data)

# Save dataset as CSV
df.to_csv("employee_data.csv", index=False)

# Display dataset
print("Dataset Created Successfully!")
print(df)

# Display dataset information
```

```
print("\nDataset Shape:", df.shape)
print("\nDataset Information:")
print(df.info())



import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# ---------------------------------------
# 1. CREATE DATASET
# ---------------------------------------

data = {
    "Age": [25, 30, 45, 28, 50, 35, 24, 42, 31, 55,
            27, 39, 48, 26, 44, 33, 29, 52, 36, 41],

    "MonthlyIncome": [20000, 35000, 70000, 25000, 85000,
                      50000, 18000, 75000, 40000, 90000,
                      22000, 60000, 80000, 21000, 72000,
                      45000, 30000, 95000, 55000, 68000],
```

```
    "YearsAtCompany": [1, 3, 15, 2, 20, 8, 1, 12, 4, 22,
                       2, 10, 18, 1, 14, 6, 3, 25, 9, 13],

    "OverTime": ["Yes", "No", "No", "Yes", "No",
                 "Yes", "Yes", "No", "Yes", "No",
                 "Yes", "No", "No", "Yes", "No",
                 "Yes", "Yes", "No", "No", "No"],

    "JobSatisfaction": [2, 3, 4, 1, 4, 3, 1, 4, 2, 4,
                        1, 3, 4, 2, 3, 2, 1, 4, 3, 4],

    "Attrition": ["Yes", "No", "No", "Yes", "No",
                  "No", "Yes", "No", "Yes", "No",
                  "Yes", "No", "No", "Yes", "No",
                  "Yes", "Yes", "No", "No", "No"]
}

df = pd.DataFrame(data)

# Save dataset
df.to_csv("employee_data.csv", index=False)

print("Dataset Created Successfully!")

# ---------------------------------------
# 2. LOAD AND EXPLORE DATASET
# ---------------------------------------

df = pd.read_csv("employee_data.csv")
```

```
print("\nFirst 5 Records:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Description:")
print(df.describe())

# ---------------------------------------
# 3. DATA VISUALIZATION
# ---------------------------------------

print("\nAttrition Count:")
print(df["Attrition"].value_counts())

df["Attrition"].value_counts().plot(kind="bar")

plt.title("Employee Attrition")
plt.xlabel("Attrition")
plt.ylabel("Number of Employees")
plt.show()

# ---------------------------------------
```

```
# 4. DATA PREPROCESSING AND ENCODING
# ----------------------------------------

# Encode categorical input column
df = pd.get_dummies(
    df,
    columns=["OverTime"],
    drop_first=True,
    dtype=int
)

# Encode target variable
df["Attrition"] = df["Attrition"].map({
    "No": 0,
    "Yes": 1
})

print("\nEncoded Dataset:")
print(df.head())

# ----------------------------------------
# 5. SEPARATE X AND Y
# ----------------------------------------

X = df.drop("Attrition", axis=1)
y = df["Attrition"]

print("\nX Shape:", X.shape)
print("Y Shape:", y.shape)
```

```
# ---------------------------------------
# 6. TRAIN-TEST SPLIT
# ---------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# ---------------------------------------
# 7. DECISION TREE MODEL TRAINING
# ---------------------------------------

model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

model.fit(X_train, y_train)

print("\nDecision Tree Model Trained Successfully!")

# ---------------------------------------
# 8. MODEL EVALUATION
```

```
# ---------------------------------------

y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

# ---------------------------------------
# 9. PREDICT NEW EMPLOYEE
# ---------------------------------------

new_employee = pd.DataFrame([{
    "Age": 26,
    "MonthlyIncome": 22000,
    "YearsAtCompany": 2,
    "JobSatisfaction": 1,
    "OverTime_Yes": 1
}])

prediction = model.predict(new_employee)
```

```
if prediction[0] == 1:
    print("\nPrediction: Employee may leave the company.")
else:
    print("\nPrediction: Employee may stay in the company.")

# ---------------------------------------
# 10. VISUALIZE DECISION TREE
# ---------------------------------------

plt.figure(figsize=(14, 8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No", "Yes"],
    filled=True,
    rounded=True
)

plt.title("Employee Attrition Decision Tree")
plt.show()
```

Exp-8 Fuzzy

```
import numpy as np
import matplotlib.pyplot as plt

# ====================================================
# PART 1: CLASSICAL SET AND FUZZY SET
# ====================================================

universe = np.array([1, 2, 3, 4, 5])

classical_set = np.array([0, 0, 0, 1, 1])
fuzzy_set = np.array([0.1, 0.3, 0.5, 0.8, 1.0])

print("\n--- PART 1: CLASSICAL AND FUZZY SETS ---")
print("Universe:", universe)
print("Classical Set:", classical_set)
print("Fuzzy Set:", fuzzy_set)



# ====================================================
# PART 2: MEMBERSHIP FUNCTIONS
# ====================================================

def triangular_mf(x, a, b, c):
    if x <= a or x >= c:
        return 0
    elif x == b:
        return 1
    elif x < b:
```

```
        return (x - a) / (b - a)
    else:
        return (c - x) / (c - b)


def trapezoidal_mf(x, a, b, c, d):
    if x <= a or x >= d:
        return 0
    elif b <= x <= c:
        return 1
    elif x < b:
        return (x - a) / (b - a)
    else:
        return (d - x) / (d - c)


def gaussian_mf(x, mean, sigma):
    return np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))


x = np.linspace(0, 10, 100)

triangular = [triangular_mf(i, 2, 5, 8) for i in x]
trapezoidal = [trapezoidal_mf(i, 2, 4, 6, 8) for i in x]
gaussian = [gaussian_mf(i, 5, 1.5) for i in x]

print("\n--- PART 2: MEMBERSHIP FUNCTIONS ---")
print("Triangular, Trapezoidal and Gaussian functions calculated.")

plt.figure()
```

```
plt.plot(x, triangular, label="Triangular")
plt.plot(x, trapezoidal, label="Trapezoidal")
plt.plot(x, gaussian, label="Gaussian")
plt.xlabel("Input")
plt.ylabel("Membership Degree")
plt.title("Membership Functions")
plt.legend()
plt.grid()
plt.show()



# ====================================================
# PART 3: FUZZY SET OPERATIONS
# ====================================================

A = np.array([0.2, 0.5, 0.8, 1.0])
B = np.array([0.6, 0.3, 0.7, 0.4])

union = np.maximum(A, B)
intersection = np.minimum(A, B)
complement = 1 - A

print("\n--- PART 3: FUZZY SET OPERATIONS ---")
print("A:", A)
print("B:", B)
print("Union:", union)
print("Intersection:", intersection)
print("Complement of A:", complement)
```

```
# ====================================================
# PART 4: ALGEBRAIC PROPERTIES
# ====================================================

C = np.array([0.4, 0.7, 0.2, 0.9])

# Commutative Law
commutative = np.allclose(
    np.maximum(A, B), np.maximum(B, A)
)

# Associative Law
associative = np.allclose(
    np.maximum(np.maximum(A, B), C),
    np.maximum(A, np.maximum(B, C))
)

# Idempotent Law
idempotent = np.allclose(
    np.maximum(A, A), A
)

# Absorption Law
absorption = np.allclose(
    np.minimum(A, np.maximum(A, B)), A
)

# De Morgan's Law
demorgan = np.allclose(
    1 - np.maximum(A, B),
```

```
    np.minimum(1 - A, 1 - B)
)

# Distributive Law
distributive = np.allclose(
    np.minimum(A, np.maximum(B, C)),
    np.maximum(np.minimum(A, B), np.minimum(A, C))
)

print("\n--- PART 4: ALGEBRAIC PROPERTIES ---")
print("Commutative:", commutative)
print("Associative:", associative)
print("Idempotent:", idempotent)
print("Absorption:", absorption)
print("De Morgan:", demorgan)
print("Distributive:", distributive)



# ====================================================
# PART 5: FUZZY RELATIONS AND MAX-MIN COMPOSITION
# ====================================================

A1 = np.array([0.2, 0.5, 0.8])
B1 = np.array([0.6, 0.3])

# Cartesian Product using MIN
R = np.minimum(A1[:, None], B1[None, :])

print("\n--- PART 5: FUZZY RELATIONS ---")
print("Cartesian Product Relation R:\n", R)
```

```
S = np.array([
    [0.7, 0.4],
    [0.5, 0.9]
])

# Max-Min Composition
T = np.zeros((3, 2))

for i in range(3):
    for j in range(2):
        for k in range(2):
            T[i, j] = max(
                T[i, j],
                min(R[i, k], S[k, j])
            )

print("Max-Min Composition:\n", T)



# ====================================================
# PART 6: FUZZY EQUIVALENCE RELATION
# ====================================================

R1 = np.array([
    [1.0, 0.7, 0.5],
    [0.7, 1.0, 0.6],
    [0.5, 0.6, 1.0]
])
```

```
# Reflexivity
reflexive = np.all(np.diag(R1) == 1)

# Symmetry
symmetric = np.allclose(R1, R1.T)

# Max-Min Composition of R1 with itself
composition = np.zeros_like(R1)

for i in range(3):
    for j in range(3):
        for k in range(3):
            composition[i, j] = max(
                composition[i, j],
                min(R1[i, k], R1[k, j])
            )

# Max-Min Transitivity
transitive = np.all(R1 >= composition)

print("\n--- PART 6: FUZZY EQUIVALENCE RELATION ---")
print("Relation Matrix:\n", R1)
print("Reflexive:", reflexive)
print("Symmetric:", symmetric)
print("Transitive:", transitive)

if reflexive and symmetric and transitive:
    print("R is a Fuzzy Equivalence Relation")
else:
    print("R is not a Fuzzy Equivalence Relation")
```

```
# ====================================================
# PART 7: EXTENSION PRINCIPLE
# ====================================================

x_values = np.array([-2, -1, 1, 2])
membership = np.array([0.3, 0.7, 0.5, 0.9])

# Mathematical Function
y_values = x_values ** 2

# Apply Extension Principle
output_set = {}

for value, degree in zip(y_values, membership):
    output_set[value] = max(
        output_set.get(value, 0), degree
    )

print("\n--- PART 7: EXTENSION PRINCIPLE ---")
print("Input Values:", x_values)
print("Membership Values:", membership)
print("Output Values:", y_values)
print("Output Fuzzy Set:", output_set)
```

Exp 3 – Adder

```
import numpy as np
import matplotlib.pyplot as plt

# ====================================================
# PART 1: HALF ADDER AND FULL ADDER TRUTH TABLES
# ====================================================

def half_adder(A, B):
    SUM = A ^ B
    CARRY = A & B
    return SUM, CARRY



def full_adder(A, B, Cin):
    SUM = A ^ B ^ Cin
    CARRY = (A & B) | (B & Cin) | (A & Cin)
    return SUM, CARRY



print("\n========== HALF ADDER ==========")

for A in [0, 1]:
    for B in [0, 1]:
        SUM, CARRY = half_adder(A, B)
        print(A, B, "->", SUM, CARRY)



print("\n========== FULL ADDER ==========")
```

```
for A in [0, 1]:
    for B in [0, 1]:
        for Cin in [0, 1]:
            SUM, CARRY = full_adder(A, B, Cin)
            print(A, B, Cin, "->", SUM, CARRY)


# ======================================================
# PART 2: ACTIVATION FUNCTIONS
# ======================================================

def sigmoid(x):
    x = np.clip(x, -500, 500)
    return 1 / (1 + np.exp(-x))


def tanh(x):
    return np.tanh(x)


def relu(x):
    return np.maximum(0, x)


def leaky_relu(x):
    return np.where(x > 0, x, 0.01 * x)


def softmax(x):
```

```
    exp_values = np.exp(x - np.max(x))
    return exp_values / np.sum(exp_values)


x = np.linspace(-5, 5, 100)

sigmoid_values = sigmoid(x)
tanh_values = tanh(x)
relu_values = relu(x)
leaky_values = leaky_relu(x)

softmax_values = softmax(np.array([2.0, 1.0, 0.1]))

print("\n========== ACTIVATION FUNCTIONS ==========")
print("Softmax Output:", softmax_values)
print("Softmax Sum:", np.sum(softmax_values))

plt.figure(figsize=(10, 6))

plt.plot(x, sigmoid_values, label="Sigmoid")
plt.plot(x, tanh_values, label="Tanh")
plt.plot(x, relu_values, label="ReLU")
plt.plot(x, leaky_values, label="Leaky ReLU")

plt.xlabel("Input")
plt.ylabel("Output")
plt.title("Activation Functions")
plt.legend()
plt.grid()
plt.show()
```

```
# ====================================================
# PART 3: ANN ARCHITECTURE AND FORWARD PROPAGATION
# ====================================================

# Half Adder Inputs
X_half = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

# Random weights
np.random.seed(42)

W1 = np.random.rand(2, 4)
b1 = np.zeros((1, 4))

W2 = np.random.rand(4, 2)
b2 = np.zeros((1, 2))

# Forward Propagation
hidden = sigmoid(np.dot(X_half, W1) + b1)
output = sigmoid(np.dot(hidden, W2) + b2)

print("\n========== FORWARD PROPAGATION ==========")
print("Input:\n", X_half)
print("Hidden Layer Output:\n", hidden)
```

```
print("Final Output:\n", output)

# Note: These are random initial predictions.
# The ANN learns in the next training sections.



# ======================================================
# PART 4: BACKPROPAGATION AND HALF ADDER TRAINING
# ======================================================

# Half Adder Expected Outputs
Y_half = np.array([
    [0, 0],
    [1, 0],
    [1, 0],
    [0, 1]
], dtype=float)


def sigmoid_derivative(x):
    return x * (1 - x)


def train_ann(X, Y, hidden_neurons, epochs, learning_rate):

    np.random.seed(42)

    # Initialize weights and biases
    W1 = np.random.randn(X.shape[1], hidden_neurons) * 0.5
    b1 = np.zeros((1, hidden_neurons))
```

```
W2 = np.random.randn(hidden_neurons, Y.shape[1]) * 0.5
b2 = np.zeros((1, Y.shape[1]))

# Training loop
for epoch in range(epochs):

    # Forward Propagation
    hidden = sigmoid(np.dot(X, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)

    # Calculate Error and Loss
    error = Y - output
    loss = np.mean(error ** 2)

    # Backpropagation
    d_output = error * sigmoid_derivative(output)

    d_hidden = (
        np.dot(d_output, W2.T)
        * sigmoid_derivative(hidden)
    )

    # Update weights and biases
    W2 += learning_rate * np.dot(hidden.T, d_output) / len(X)

    b2 += learning_rate * np.sum(
        d_output, axis=0, keepdims=True
    ) / len(X)
```

```
        W1 += learning_rate * np.dot(X.T, d_hidden) / len(X)

        b1 += learning_rate * np.sum(
            d_hidden, axis=0, keepdims=True
        ) / len(X)

        if epoch % 10000 == 0:
            print("Epoch:", epoch, "Loss:", loss)

    # Final predictions after training
    hidden = sigmoid(np.dot(X, W1) + b1)
    predictions = sigmoid(np.dot(hidden, W2) + b2)

    return predictions, loss


print("\n========== HALF ADDER ANN TRAINING ==========")

half_predictions, half_loss = train_ann(
    X_half,
    Y_half,
    hidden_neurons=4,
    epochs=30000,
    learning_rate=5.0
)

print("\nHalf Adder Predicted Outputs:")

for i in range(len(X_half)):
    predicted = (half_predictions[i] >= 0.5).astype(int)
```

```
    print(
        X_half[i].astype(int),
        "->",
        predicted
    )

print("\nActual Half Adder Outputs:")
print(Y_half.astype(int))

print("Half Adder Final Loss:", half_loss)



# ======================================================
# PART 5: FULL ADDER ANN TRAINING AND TESTING
# ======================================================

# Full Adder Inputs
X_full = np.array([
    [0, 0, 0],
    [0, 0, 1],
    [0, 1, 0],
    [0, 1, 1],
    [1, 0, 0],
    [1, 0, 1],
    [1, 1, 0],
    [1, 1, 1]
], dtype=float)
```

```
# Full Adder Expected Outputs
Y_full = np.array([
    [0, 0],
    [1, 0],
    [1, 0],
    [0, 1],
    [1, 0],
    [0, 1],
    [0, 1],
    [1, 1]
], dtype=float)


print("\n========== FULL ADDER ANN TRAINING ==========")

full_predictions, full_loss = train_ann(
    X_full,
    Y_full,
    hidden_neurons=8,
    epochs=50000,
    learning_rate=5.0
)


print("\nFull Adder Predicted Outputs:")

for i in range(len(X_full)):

    predicted = (full_predictions[i] >= 0.5).astype(int)
```

```
    print(
        X_full[i].astype(int),
        "->",
        predicted
    )


print("\nActual Full Adder Outputs:")
print(Y_full.astype(int))

print("Full Adder Final Loss:", full_loss)


# ======================================================
# PART 6: FINAL ACCURACY CHECK
# ======================================================

half_result = (half_predictions >= 0.5).astype(int)
full_result = (full_predictions >= 0.5).astype(int)

half_accuracy = np.mean(half_result == Y_half) * 100
full_accuracy = np.mean(full_result == Y_full) * 100

print("\n========== FINAL RESULTS ==========")

print("Half Adder Accuracy:", half_accuracy, "%")
print("Full Adder Accuracy:", full_accuracy, "%")

print("\nExperiment 3 Completed Successfully!")
```

Exp 4 – Single Layer Perceptron

```
import numpy as np

# ====================================================
# PART 1: STEP ACTIVATION FUNCTION
# ====================================================

def step_function(x):
    return 1 if x >= 0 else 0



# ====================================================
# PART 2: PERCEPTRON LEARNING ALGORITHM
# ====================================================

def train_perceptron(X, Y, epochs=20):

    # Initialize weights and bias
    weights = np.zeros(X.shape[1])
    bias = 0

    learning_rate = 0.1

    # Training
    for epoch in range(epochs):

        total_error = 0

        for i in range(len(X)):
```

```
            # Calculate weighted sum
            z = np.dot(X[i], weights) + bias

            # Predict output
            prediction = step_function(z)

            # Calculate error
            error = Y[i] - prediction

            # Update weights and bias
            weights += learning_rate * error * X[i]
            bias += learning_rate * error

            total_error += abs(error)

        print("Epoch:", epoch + 1,
              "Total Error:", total_error)

        # Stop if all predictions are correct
        if total_error == 0:
            break

    return weights, bias


# ====================================================
# PART 3: TESTING PERCEPTRON
# ====================================================
```

```
def test_perceptron(X, Y, weights, bias):

    correct = 0

    print("\nInput -> Predicted | Actual")

    for i in range(len(X)):

        z = np.dot(X[i], weights) + bias
        prediction = step_function(z)

        print(
            X[i], "->", prediction,
            "|", Y[i]
        )

        if prediction == Y[i]:
            correct += 1

    accuracy = (correct / len(X)) * 100

    print("Accuracy:", accuracy, "%")

    return accuracy


# ====================================================
# PART 4: AND GATE
# ====================================================
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

Y_and = np.array([0, 0, 0, 1])

print("\n========== AND GATE ==========")

weights_and, bias_and = train_perceptron(
    X, Y_and, epochs=20
)

print("\nFinal Weights:", weights_and)
print("Final Bias:", bias_and)

test_perceptron(
    X, Y_and, weights_and, bias_and
)



# ================================================
# PART 5: OR GATE
# ================================================

Y_or = np.array([0, 1, 1, 1])

print("\n========== OR GATE ==========")
```

```
weights_or, bias_or = train_perceptron(
    X, Y_or, epochs=20
)

print("\nFinal Weights:", weights_or)
print("Final Bias:", bias_or)

test_perceptron(
    X, Y_or, weights_or, bias_or
)



# ======================================================
# PART 6: XOR GATE
# ======================================================

Y_xor = np.array([0, 1, 1, 0])

print("\n========== XOR GATE ==========")

weights_xor, bias_xor = train_perceptron(
    X, Y_xor, epochs=20
)

print("\nFinal Weights:", weights_xor)
print("Final Bias:", bias_xor)

xor_accuracy = test_perceptron(
    X, Y_xor, weights_xor, bias_xor
```

```
)



# ==================================================
# PART 7: FINAL CONCLUSION
# ==================================================

print("\n========== FINAL CONCLUSION ==========")

print("AND Gate: Linearly Separable")
print("OR Gate: Linearly Separable")
print("XOR Gate: Not Linearly Separable")

print("\nExperiment 4 Completed Successfully!")
```

Experiment 10 – ANFIS

```
import pandas as pd
import numpy as np

np.random.seed(42)

n = 300

density = np.random.randint(5, 101, n)
speed = np.random.randint(10, 101, n)
time = np.random.randint(0, 24, n)

# Generate congestion score using traffic conditions
congestion = (
    0.55 * density
    + 0.30 * (100 - speed)
    + 0.15 * np.where(
        (time >= 7) & (time <= 10) |
        (time >= 17) & (time <= 20),
        100, 30
    )
)

# Add small realistic variation
congestion += np.random.normal(0, 4, n)

congestion = np.clip(congestion, 0, 100)

df = pd.DataFrame({
```

```
    "Density": density,
    "Speed": speed,
    "Time": time,
    "Congestion": congestion.round(2)
})

df.to_csv("traffic_data.csv", index=False)

print("Dataset created successfully!")
print(df.head())
print("\nDataset shape:", df.shape)
print("\nDataset information:")
print(df.describe())



import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

from itertools import product
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

# ========================================
# STEP 1: GENERATE TRAFFIC DATASET
# ========================================
```

```
np.random.seed(42)
torch.manual_seed(42)

n = 300

density = np.random.randint(5, 101, n)
speed = np.random.randint(10, 101, n)
time = np.random.randint(0, 24, n)

rush_hour = (
    ((time >= 7) & (time <= 10)) |
    ((time >= 17) & (time <= 20))
)

congestion = (
    0.55 * density
    + 0.30 * (100 - speed)
    + 0.15 * np.where(rush_hour, 100, 30)
)

congestion += np.random.normal(0, 4, n)
congestion = np.clip(congestion, 0, 100)

df = pd.DataFrame({
    "Density": density,
    "Speed": speed,
    "Time": time,
    "Congestion": congestion.round(2)
})
```

```
df.to_csv("traffic_data.csv", index=False)

print("Dataset created successfully!")
print(df.head())
print("Dataset Shape:", df.shape)



# ========================================
# STEP 2: DATA PREPROCESSING
# ========================================

X = df[["Density", "Speed", "Time"]].values
y = df[["Congestion"]].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

x_scaler = MinMaxScaler()
y_scaler = MinMaxScaler()

X_train = x_scaler.fit_transform(X_train)
X_test = x_scaler.transform(X_test)

y_train = y_scaler.fit_transform(y_train)
y_test = y_scaler.transform(y_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
```

```
y_train = torch.tensor(y_train, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32)

print("\nTraining Input Shape:", X_train.shape)
print("Testing Input Shape:", X_test.shape)
print("Preprocessing completed!")



# ========================================
# STEP 3: ANFIS MODEL
# ========================================

class ANFIS(nn.Module):

    def __init__(self):
        super().__init__()

        # Layer 1: Gaussian membership functions
        self.centers = nn.Parameter(
            torch.tensor([
                [0.0, 0.5, 1.0],
                [0.0, 0.5, 1.0],
                [0.0, 0.5, 1.0]
            ], dtype=torch.float32)
        )

        self.sigmas = nn.Parameter(
            torch.ones(3, 3) * 0.3
        )
```

```
    # 27 rules, 3 coefficients + 1 bias per rule
    self.consequent = nn.Parameter(
        torch.zeros(27, 4)
    )

def forward(self, x):

    # Layer 1: Fuzzification
    membership = torch.exp(
        -((x.unsqueeze(2) - self.centers) ** 2)
        / (2 * self.sigmas ** 2)
    )

    # Layer 2: Rule firing strengths
    firing = (
        membership[:, 0, :, None, None]
        * membership[:, 1, None, :, None]
        * membership[:, 2, None, None, :]
    )

    firing = firing.reshape(x.shape[0], 27)

    # Layer 3: Normalization
    normalized = firing / (
        firing.sum(dim=1, keepdim=True) + 1e-8
    )

    # Layer 4: Sugeno consequent outputs
    rule_input = torch.cat(
```

```
            [x, torch.ones(x.shape[0], 1)], dim=1
        )

        rule_output = rule_input @ self.consequent.T

        # Layer 5: Final output
        output = torch.sum(
            normalized * rule_output,
            dim=1,
            keepdim=True
        )

        return output


model = ANFIS()

print("\nANFIS model created successfully!")
print("Number of inputs: 3")
print("Membership functions per input: 3")
print("Number of fuzzy rules:", 3 ** 3)
print("Number of ANFIS layers: 5")


# ========================================
# STEP 4: HYBRID LEARNING
# ========================================

# Gradient descent updates premise parameters
optimizer = torch.optim.Adam(
```

```
    [model.centers, model.sigmas],
    lr=0.005
)

criterion = nn.MSELoss()

epochs = 500
regularization = 1e-4


def estimate_consequents():

    # Calculate normalized firing strengths
    with torch.no_grad():

        membership = torch.exp(
            -((X_train.unsqueeze(2) - model.centers) ** 2)
            / (2 * model.sigmas ** 2)
        )

        firing = (
            membership[:, 0, :, None, None]
            * membership[:, 1, None, :, None]
            * membership[:, 2, None, None, :]
        ).reshape(X_train.shape[0], 27)

        normalized = firing / (
            firing.sum(dim=1, keepdim=True) + 1e-8
        )
```

```
        # Construct Sugeno input
        rule_input = torch.cat(
            [X_train, torch.ones(X_train.shape[0], 1)],
            dim=1
        )

        # Least squares design matrix
        design = (
            normalized.unsqueeze(2)
            * rule_input.unsqueeze(1)
        ).reshape(X_train.shape[0], 27 * 4)

        # Ridge-regularized least squares
        identity = torch.eye(design.shape[1])

        coefficients = torch.linalg.solve(
            design.T @ design
            + regularization * identity,
            design.T @ y_train
        )

        model.consequent.copy_(
            coefficients.reshape(27, 4)
        )



print("\nHybrid training started...")

for epoch in range(epochs):
```

```
    # Forward pass: Least Squares Estimation
    estimate_consequents()

    # Backward pass: Gradient Descent
    predictions = model(X_train)

    loss = criterion(predictions, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Keep membership widths positive
    with torch.no_grad():
        model.sigmas.clamp_(min=0.05)

    if (epoch + 1) % 100 == 0:
        print(
            f"Epoch [{epoch+1}/{epochs}], "
            f"Loss: {loss.item():.6f}"
        )

# Final LSE estimation after premise updates
estimate_consequents()

print("\nHybrid training completed!")



# ========================================
# STEP 5: MODEL EVALUATION
```

```
# ========================================

model.eval()

with torch.no_grad():
    test_predictions = model(X_test).numpy()

predicted = y_scaler.inverse_transform(test_predictions)
actual = y_scaler.inverse_transform(y_test.numpy())

mae = mean_absolute_error(actual, predicted)
mse = mean_squared_error(actual, predicted)
rmse = np.sqrt(mse)

print("\nModel Evaluation:")
print("MAE:", round(mae, 3))
print("MSE:", round(mse, 3))
print("RMSE:", round(rmse, 3))

print("\nActual vs Predicted:")

for i in range(5):
    print(
        f"Actual: {actual[i][0]:.2f}, "
        f"Predicted: {predicted[i][0]:.2f}"
    )



# ========================================
# STEP 6: VISUALIZATION
```

```
# ========================================

plt.figure(figsize=(8, 5))

plt.scatter(actual, predicted, alpha=0.7)

plt.plot(
    [0, 100], [0, 100],
    linestyle="--",
    label="Perfect Prediction"
)

plt.xlabel("Actual Congestion")
plt.ylabel("Predicted Congestion")
plt.title("ANFIS: Actual vs Predicted Congestion")
plt.legend()
plt.grid()

plt.savefig("anfis_prediction.png")
plt.show()



# ========================================
# STEP 7: NEW TRAFFIC PREDICTION
# ========================================

# Density = 80, Speed = 20, Time = 18
new_data = np.array([[80, 20, 18]])

new_scaled = x_scaler.transform(new_data)
```

```
new_tensor = torch.tensor(
    new_scaled, dtype=torch.float32
)

model.eval()

with torch.no_grad():
    result = model(new_tensor).numpy()

result = y_scaler.inverse_transform(result)

score = float(np.clip(result[0][0], 0, 100))

print("\nNew Traffic Prediction:")
print("Density: 80")
print("Speed: 20 km/h")
print("Time: 18:00")

print("Predicted Congestion:", round(score, 2))

if score >= 70:
    print("Traffic Status: HIGH CONGESTION")
elif score >= 40:
    print("Traffic Status: MODERATE CONGESTION")
else:
    print("Traffic Status: LOW CONGESTION")

print("\nExperiment 10 completed successfully!")
```